

Computer Networks



Overview of the Internet

Protocol layering (Book 1 Ch 1)

Application Layer

- Application-Layer Paradigms
- Client-server applications
 - World Wide Web and HTTP
 - FTP
 - Electronic Mail
 - DNS
- Peer-to-peer paradigm
 - P2P Networks
- Case study: BitTorrent (Book 1 Ch 2)

World Wide Web

- The **World Wide Web (WWW or Web)** is a repository of information.
- Documents, called **web pages**, are:
 - Distributed all over the world
 - Linked to related documents
- The Web was proposed by **Tim Berners-Lee** in **1989** at CERN.
- Original purpose:
 - Allow researchers at different locations to access each other's research.
- The commercial Web started in the **early 1990s**.

Why Did the Web Grow So Rapidly?

The popularity and growth of the Web are based on two key characteristics:

1. Distributed

- Web pages are stored on servers throughout the world.
- Any web server can add new pages to the Web.
- Growth does not overload a small number of central servers.

2. Linked

- One web page can refer to another page.
- The linked page may be stored on another server anywhere in the world.

From Hypertext to Hypermedia

Hypertext

A system of linked text documents in which a link can be used to retrieve another document.

- The Web implemented the idea of hypertext electronically.
- Clicking a link retrieves the linked document.
- Modern web pages contain more than text.

Hypermedia

Linked information may include:

- Text
- Images
- Audio
- Video

Architecture of the World Wide Web

WWW Architecture

The Web is a **distributed client-server service**.

- A **client** uses a browser to access a service.
- The service is provided by a **web server**.
- Web services are distributed over many locations called **sites**.
- Each site contains one or more **web pages**.
- A web page may contain links to pages in the same or other sites.
- Every web page is a file with its own **name and address**.

Simple and Composite Web Pages

Simple Web Page

A web page with no links to other web pages.

Composite Web Page

A web page containing one or more links to other web pages.

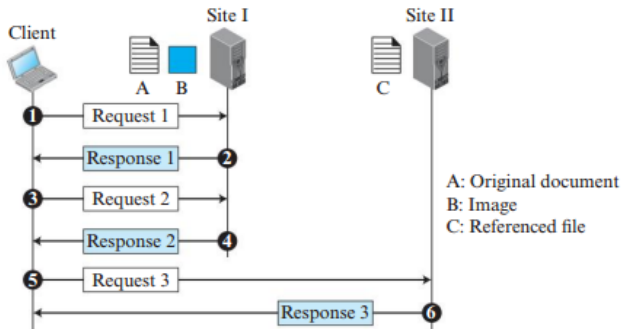
- Linked pages may be stored at the same site or different sites.
- Each linked resource remains an **independent file**.

Example 2.2: Retrieving a Scientific Document

Assume a scientific document contains:

- One reference to another text file
- One reference to a large image
- **File A:** Main document – Site I
- **File B:** Large image – Site I
- **File C:** Referenced text file – Site II

Figure 2.8 *Example 2.2*



Three Independent Transactions

① Transaction 1

- Retrieve File A, the main document.
- File A contains references to Files B and C.

② Transaction 2

- User clicks the image reference.
- File B is retrieved.

③ Transaction 3

- User clicks the text-file reference.
- File C is retrieved.

Three files $\Rightarrow$ Three transactions

Important Point: Independent Web Pages

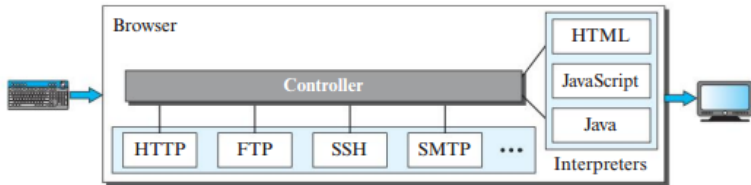
- Files A, B, and C are **independent web pages**.
- Each file has:
 - An independent name
 - An independent address
- Files A and B are stored at the same site.
- However, they require separate transactions.
- A reference to File B or C inside File A does **not** make them part of File A.
- Any file can be retrieved independently using its own address.

A web browser usually consists of three components:

- 1 **Controller**
- 2 **Client Protocols**
- 3 **Interpreters**



Figure 2.9 *Browser*



Browser Components

Controller

- Receives input from the keyboard or mouse.
- Uses a client protocol to access the document.
- Selects an interpreter to display the document.

Client Protocols

Examples: HTTP and FTP

Interpreters

Examples: HTML, Java, and JavaScript

- Web pages are stored at the server.
- When a request arrives:
 - ① The server identifies the requested document.
 - ② The document is retrieved.
 - ③ A copy is sent to the client.

Improving Server Efficiency

- **Caching:** Frequently requested files are stored in memory.
- **Multithreading / Multiprocessing:** Multiple requests can be answered simultaneously.

Uniform Resource Locator (URL)

A web page needs a unique identifier.

Four Components of a URL

- 1 Protocol
- 2 Host
- 3 Port
- 4 Path

URL Formats

`protocol://host/path`

`protocol://host:port/path`

URL Components

- Protocol** Identifies the client-server application used to access the resource.
- Host** Identifies the server using an IP address or domain name.
- Port** A 16-bit integer identifying the application process.
- Path** Identifies the location and name of the file.

Example

`http://www.mhhe.com/compsci/forouzan/`

Example 2.3: Analyzing a URL

Consider:

`http://www.mhhe.com/compsci/forouzan/`

Component	Value
Protocol	http
Host	www.mhhe.com
Port	Default HTTP port 80
Path	/compsci/forouzan/

URL = Protocol + Host + Port + Path

Types of Web Documents

Documents on the Web are grouped into three categories:

- 1 **Static Documents**
- 2 **Dynamic Documents**
- 3 **Active Documents**

Main Difference

When and where is the content created or executed?

Static Documents

Definition

A static document is a fixed-content document created and stored on a server.

- Content is determined when the file is created.
- The client receives only a copy of the document.
- The user cannot change the original document on the server.
- The same stored content is returned for each request.

Languages

HTML, XML, XSL, and XHTML

Dynamic Documents

Definition

A dynamic document is created by the web server whenever a browser requests it.

- 1 A request arrives.
- 2 The server runs a program or script.
- 3 The program creates the document.
- 4 The result is sent to the browser.

Key Characteristic

A fresh document is created for each request; therefore, the content may change.

Dynamic Document: Example and Technologies

Example: Current Date and Time

- Time changes continuously.
- The server runs a program when the request arrives.
- The current result is generated and returned to the client.

Technologies

- CGI – Common Gateway Interface
- JSP – Java Server Pages
- ASP – Active Server Pages
- ColdFusion

Definition

An active document contains a program or script that is sent by the server and executed at the client site.

- Required when processing must occur at the client.
- Examples:
 - Animated graphics
 - User interaction
 - Interactive applications
- Technologies:
 - Java applets
 - JavaScript

Static vs Dynamic vs Active Documents

Feature	Static	Dynamic	Active
Content	Fixed	Generated	Interactive
Created	Before request	At request	Before request
Processing	None	Server	Client
Example	HTML	Current time	JavaScript

Easy Way to Remember

- **Static** – Send the stored file
- **Dynamic** – Server creates the result
- **Active** – Client runs the code

HyperText Transfer Protocol (HTTP)

Definition

The **HyperText Transfer Protocol (HTTP)** defines how client-server programs communicate to retrieve web pages from the Web.

- An **HTTP client** sends a request.
- An **HTTP server** returns a response.
- The server uses well-known port number **80**.
- The client uses a **temporary port number**.

HTTP Client Request → HTTP Server

HTTP Client ← HTTP Server Response

HTTP Uses TCP

- HTTP uses the services of **TCP**.
- TCP is:
 - Connection-oriented
 - Reliable
- Before an HTTP transaction:
 - A TCP connection must be established.
- After the transaction:
 - The connection may be terminated.
- HTTP does not need to handle:
 - Message errors
 - Message loss

Why?

TCP provides reliable delivery and handles these problems.

Why Are Multiple HTTP Requests Needed?

- A web page may contain references to several objects.
- Examples:
 - Images
 - Text files
 - Other web pages
- Each object requires an HTTP request and response.
- If objects are located on **different servers**:
 - A separate TCP connection is required for each server.
- If objects are located on the **same server**, two strategies are possible.

Two Types of HTTP Connections

If multiple objects are stored on the same server, HTTP can use:

Nonpersistent Connection

A new TCP connection is created for each request-response pair.

Persistent Connection

One TCP connection is created and used to retrieve multiple objects.

- Before HTTP/1.1:
 - Nonpersistent connections were specified.
- HTTP/1.1:
 - Persistent connections are the default.

Nonpersistent Connections

Definition

In a nonpersistent connection, one TCP connection is created for each request-response pair.

- 1 The client opens a TCP connection and sends a request.
- 2 The server sends the response and closes the connection.
- 3 The client reads the data until it encounters an end-of-file marker.
- 4 The client then closes the connection.

Nonpersistent Connection: Multiple Objects

Suppose a web page contains links to N different pictures.

- Main document:

1 connection

- N pictures:



$$\text{Total TCP Connections} = N + 1$$

Problem

Every connection must be separately opened and closed.

Overhead of Nonpersistent Connections

The nonpersistent strategy creates high overhead.

- For N linked objects:

$N + 1$ connections are required

- Each connection requires:
 - Connection establishment
 - Buffer allocation
 - Variable allocation
 - Connection termination
- The burden is especially high on the server.

Main Problem

Repeated connection establishment and termination waste time and resources.

Example 2.4: Nonpersistent Connection

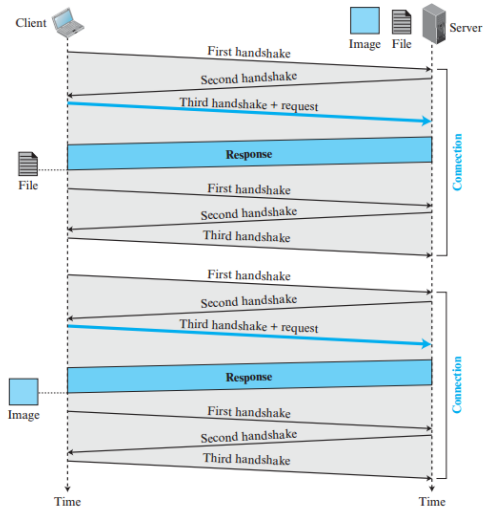
The client needs to retrieve:

- One text file
- One linked image

Both objects are located on the **same server**.

Two Objects $\Rightarrow$ Two TCP Connections

Figure 2.10 Example 2.4



Example 2.4: Connection Overhead

For each TCP connection:

- At least three handshake messages are required to establish the connection.
- The HTTP request may be sent with the third message.
- After establishment:
 - The object is transferred.
- Additional handshake messages are required to terminate the connection.

For two objects:

2 Connection Establishments + 2 Connection Terminations

Why Does the Overhead Become Significant?

Suppose a web page contains 10 or 20 objects.

- Each object requires a separate TCP connection.
- Each connection introduces:
 - Round-trip time for connection establishment
 - Round-trip time for connection termination
 - Additional buffers
 - Additional variables
- These delays accumulate.

Result

Nonpersistent HTTP can create significant time and resource overhead.

Definition

In a persistent connection, the server leaves the TCP connection open for additional requests after sending a response.

- HTTP/1.1 uses persistent connections by default.
- Multiple objects can be retrieved using the same TCP connection.
- The connection may be closed:
 - At the client's request
 - When a time-out occurs

How Does a Persistent Connection Know the End of Data?

Normally, the sender includes the **length of the data** with each response.

- The client knows exactly how much data to read.
- However, the length may be unknown when a document is:
 - Created dynamically
 - Created actively
- In such cases:
 - 1 The server informs the client that the length is unknown.
 - 2 The server sends the data.
 - 3 The server closes the connection.

Meaning of Connection Closure

The client recognizes the connection closure as the end of the data.

Advantages of Persistent Connections

Persistent connections save both time and resources.

- Only one TCP connection is needed for multiple requests.
- Only one set of resources is required at each site:
 - Buffers
 - Variables
- Saves round-trip time for repeated:
 - Connection establishment
 - Connection termination

One Connection $\Rightarrow$ Multiple Requests and Responses

Example 2.5: Persistent Connection

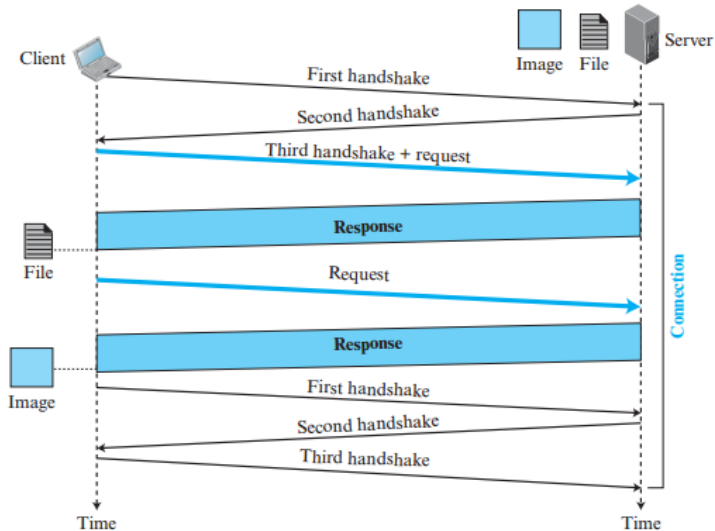
The scenario is the same as Example 2.4:

- One text file
- One linked image
- Both stored on the same server

However:

- Only one connection is established.
- The request for the image is sent separately.
- The same connection is reused.
- Only one connection termination is required.

Figure 2.11 Example 2.5



Nonpersistent vs Persistent Connections

Feature	Nonpersistent	Persistent
TCP Connections	One per object	One for multiple objects
Connection Reuse	No	Yes
Overhead	High	Low
Buffers and Variables	Allocated repeatedly	One set can be reused
Default	Before HTTP/1.1	HTTP/1.1

HTTP Message Formats

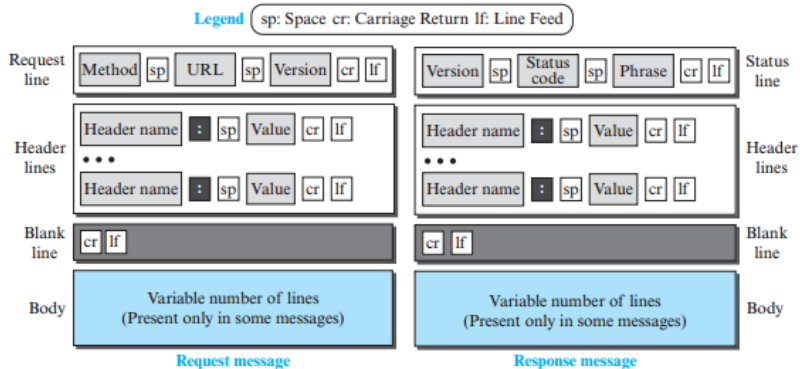
HTTP defines two types of messages:

- 1 **Request Message**
- 2 **Response Message**

Each message contains four sections.

- The first section differs:
 - Request message: **Request line**
 - Response message: **Status line**
- The remaining sections have similar names.

Figure 2.12 *Formats of the request and response messages*



HTTP Request Message Structure

An HTTP request message contains:

- 1 Request line
- 2 Zero or more header lines
- 3 Blank line
- 4 Optional body



General Structure

Request Line

Header Line(s)

Blank Line

Body (Optional)

The Request Line

The first line of a request message is called the **request line**. It contains three fields:

Method	URL	Version
--------	-----	---------

- Fields are separated by one space.
- The line ends with:
 - Carriage return
 - Line feed

Example

```
GET /index.html HTTP/1.1
```

HTTP Methods

Method	Action
GET	Requests a document from the server
HEAD	Requests information about a document, but not the document itself
PUT	Sends a document from the client to the server
POST	Sends information from the client to the server
TRACE	Echoes the incoming request
DELETE	Removes a web page
CONNECT	Reserved; may be used by proxy servers
OPTIONS	Inquires about available options

GET and HEAD Methods

GET

- Most commonly used HTTP method.
- Requests a document from the server.
- The request body is normally empty.

HEAD

- Requests information about a web page.
- Does not request the actual document.
- Can determine when a page was last modified.
- Can test the validity of a URL.
- The response body is empty.

PUT and POST Methods

PUT

- Considered the inverse of GET.
- Sends a new document from the client to the server.
- Requires permission.

POST

- Sends information from the client to the server.
- Information may be:
 - Added to a web page
 - Used to modify a web page

Other HTTP Methods

TRACE Asks the server to echo the request for debugging.

DELETE Deletes a web page if the client has permission.

CONNECT Originally reserved; may be used by proxy servers.

OPTIONS Requests information about the available options or properties of a resource.

URL and Version Fields

The request line contains:

Method	URL	Version
--------	-----	---------

URL

Defines the address and name of the requested web page.

Version

Identifies the version of the HTTP protocol.

Example:

HTTP/1.1

Request Header Lines

After the request line, there may be zero or more request header lines.

- Headers send additional information from the client to the server.
- Example:
 - The client can specify a preferred document format.

Header Format

Header Name: Header Value

Example

Accept-Language: en

Common HTTP Request Headers

Header	Description
User-Agent	Identifies the client program
Accept	Media format the client can accept
Accept-Charset	Character set the client can handle
Accept-Encoding	Encoding scheme the client can handle
Accept-Language	Language the client can accept
Authorization	Permissions of the client
Host	Host and port information
Date	Current date
Upgrade	Preferred communication protocol
Cookie	Returns a cookie to the server
If-Modified-Since	Requests the file only if modified since a specified date

Request Message Body

The body is optional in an HTTP request.

- The body is normally empty for:

- GET
- HEAD

- The body may contain data for:

- PUT
- POST

- Examples of body content:

- A comment
- Form data
- A file to be published
- A web page to be uploaded

HTTP Response Message Structure

An HTTP response message contains:

- 1 Status line
- 2 Zero or more header lines
- 3 Blank line
- 4 Optional body



General Structure

Status Line

Header Line(s)

Blank Line

Body

The Status Line

The first line of a response message is called the **status line**. It contains three fields:

Version	Status Code	Status Phrase
---------	-------------	---------------

Example

HTTP/1.1 200 OK

- Version: HTTP/1.1
- Status code: 200
- Status phrase: OK

HTTP Status Code Categories

Range	Meaning
100–199	Informational
200–299	Successful request
300–399	Redirection
400–499	Client-side error
500–599	Server-side error

Remember

$1xx \rightarrow 2xx \rightarrow 3xx \rightarrow 4xx \rightarrow 5xx$

Info $\rightarrow$ Success $\rightarrow$ Redirect $\rightarrow$ Client Error $\rightarrow$ Server Error

Response Header Lines

After the status line, there may be zero or more response header lines.

- Headers send additional information from the server to the client.
- They may describe:
 - The server
 - The document
 - The encoding
 - The language
 - The document length
 - The media type



Header Format

Header Name: Header Value

Common HTTP Response Headers

Header	Description
Date	Shows the current date
Upgrade	Preferred communication protocol
Server	Information about the server
Set-Cookie	Asks the client to save a cookie
Content-Encoding	Encoding scheme
Content-Language	Language of the content
Content-Length	Length of the document
Content-Type	Media type
Location	Redirects the client to another site
Accept-Ranges	Server accepts requested byte ranges
Last-Modified	Date and time of the last change

Response Message Body

- The body contains the document sent from the server to the client.
- The body may contain:
 - HTML
 - Images
 - Other web resources
- The body may be absent in some responses.

Important

The response body may be empty for certain error messages or responses such as HEAD and conditional responses.

Example 2.6: Retrieving a Document

The client uses the **GET** method to retrieve an image.

- Path:

`/usr/bin/image1`

- Request line contains:

- GET method
- URL
- HTTP version 1.1

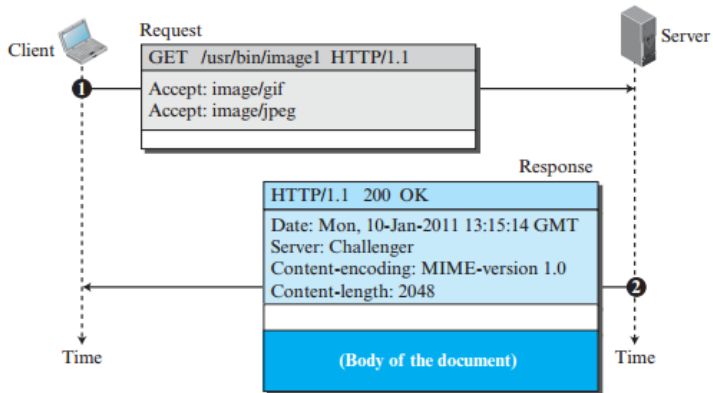


- The client can accept:

- GIF images
- JPEG images

- The request has no body.

Figure 2.13 *Example 2.6*



Example 2.6: Response Message

The response contains:

- A status line
- Four header lines describing:
 - Date
 - Server
 - Content encoding
 - Document length
- The image follows in the response body.

Status Line + Headers + Image Body

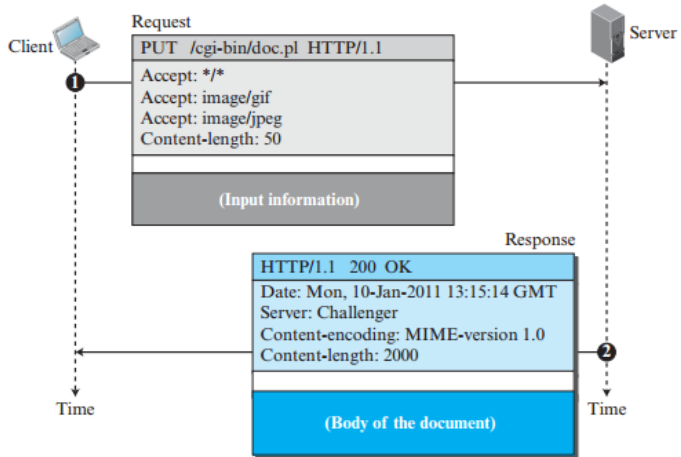
Example 2.7: Posting a Web Page

The client wants to send a web page to the server.

- The **PUT** method is used.
- The request line contains:
 - PUT
 - URL
 - HTTP/1.1
- Four request header lines are included.
- The request body contains the web page to be posted.
- The server returns a response message.



Figure 2.14 Example 2.7



Conditional Request

Definition

A client can include a condition in an HTTP request.

The server:

- Sends the requested web page if the condition is satisfied.
- Otherwise, informs the client that the condition was not satisfied.

Common Condition

Request the page only if it has been modified after a specified date and time.

If-Modified-Since Header

The client can use:

If-Modified-Since

- The client specifies a date and time.
- The server checks the last modification time of the file.
- If modified:
 - The document is sent.
- If not modified:
 - The document is not sent again.

Benefit

Avoids unnecessary transfer of unchanged documents.

Example 2.8: Conditional Request

```
GET http://www.commonServer.com/information/file1 HTTP/1.1
If-Modified-Since: Thu, Sept 04 00:00:00 GMT
```

```
HTTP/1.1 304 Not Modified
Date: Sat, Sept 06 08:16:22 GMT
Server: commonServer.com
```

Result

- Status code: **304 Not Modified**
- The requested file has not changed.
- The response body is empty.

Why Are Cookies Needed?

The original Web was designed as a **stateless system**.

Client Request → Server Response → Relationship Ends

This design worked well for retrieving public documents.

Problem

Modern web applications need to remember information about clients.

Applications That Need Memory

Modern websites may need to remember users for:

- Electronic shopping
 - Shopping carts
 - Selected items
- Access control
 - Registered users only
- Web portals
 - User-selected pages and preferences
- Advertising
 - User interests and browsing behavior

Solution

The **cookie mechanism** was created to provide state.

Creating and Storing Cookies

The basic cookie creation process has three steps:

- 1 The server receives a request and stores information about the client.
- 2 The server includes a cookie in the response.
- 3 The browser receives and stores the cookie.

Possible stored information includes:

- Client domain
- Cookie content
- Registration number
- Timestamp
- Other implementation-specific information

Cookie Creation: Server to Browser

The server includes the cookie in the HTTP response.

Response Header

```
Set-Cookie:  cookie-value
```

The browser:

- Receives the response.
- Stores the cookie.
- Organizes stored cookies by server domain name.

Server $\xrightarrow{\text{Set-Cookie}}$ Browser

Using a Stored Cookie

When the client sends another request:

- 1 The browser checks its cookie storage.
- 2 It looks for a cookie belonging to the requested server.
- 3 If found, the browser includes the cookie in the request.
- 4 The server recognizes the returning client.

Request Header

```
Cookie:  cookie-value
```

Browser $\xrightarrow{\text{Cookie}}$ Server

Cookies in Electronic Shopping

An electronic store can use cookies to maintain a shopping cart.

- ① The client selects an item.
- ② The item is added to the cart.
- ③ Cookie-related information identifies the shopping session.
- ④ More selected items are added to the same cart.
- ⑤ At checkout, the total charge is calculated.

Purpose

The server can associate multiple requests with the same shopper.

Cookies for Registered Users and Portals

Registered Users

- A cookie is sent when the client first registers.
- During later visits, the cookie identifies the returning client.
- Access can be restricted to clients with the correct cookie.

Web Portals

- A user selects preferred web pages or content.
- The preference is associated with a cookie.
- On future visits, the server restores the user's preferences.

Cookies and Advertising

Advertising agencies may use cookies to track interactions.

- An advertisement may be supplied by an external advertising server.
- The advertising server may send a cookie containing a user ID.
- Future interactions can be added to a database.
- The database may create a profile of the user's web behavior and interests.

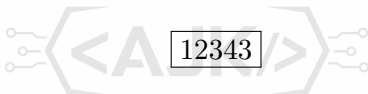
Privacy Concern

Tracking users across interactions has made the use of cookies controversial.

Shopping Cart Cookie Example

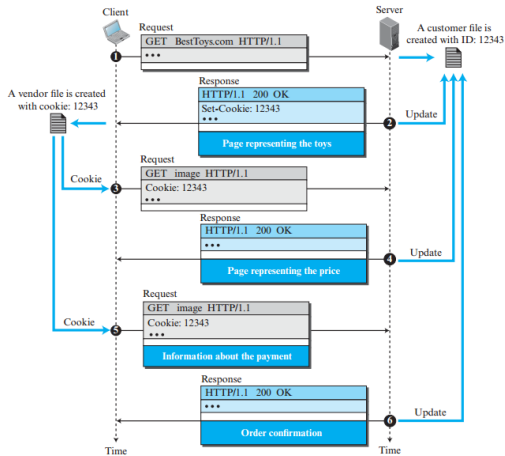
Suppose a browser sends a request to the **BestToys** server.

- 1 The server creates an empty shopping cart.
- 2 The server assigns the cart an ID:



- 3 The server sends:
 - Images of available toys
 - A Set-Cookie header containing 12343
- 4 The browser stores the cookie.

Figure 2.15 Example 2.9



Shopping Cart: Selecting an Item

When the shopper selects a toy:

- 1 The browser sends a new request.
- 2 The request contains:

Cookie: 12343

- 3 The server recognizes the returning shopper.
- 4 It finds shopping cart 12343.
- 5 The selected toy is added to that cart.

Key Idea

The cookie connects separate HTTP requests to the same shopping session.

Shopping Cart: Completing the Purchase

- 1 The server sends the total price.
- 2 The shopper provides payment information.
- 3 The browser sends another request with:



- 4 The server:
 - Identifies the cart
 - Accepts the order
 - Processes the payment
 - Sends confirmation

Definition

A proxy server is a computer that keeps copies of responses to recent requests.

Basic operation:

- ➊ The HTTP client sends a request to the proxy server.
- ➋ The proxy checks its cache.
- ➌ If the response is cached:
 - It is returned directly to the client.
- ➍ Otherwise:
 - The proxy contacts the target server.

Proxy Server: Cache Miss

When the requested response is not in the cache:

- 1 Client sends request to proxy.
- 2 Proxy checks its cache.
- 3 No cached response is found.
- 4 Proxy sends the request to the target server.
- 5 Target server returns the response.
- 6 Proxy stores a copy.
- 7 Proxy sends the response to the client.

Client → Proxy → Target Server

Advantages of a Proxy Server

A proxy server can:

- Reduce the load on the original server.
- Decrease Internet traffic.
- Improve latency.
- Serve frequently requested content locally.

Requirement

The client must be configured to access the proxy server instead of directly accessing the target server.

A Proxy Acts as Both Server and Client

Proxy as a Server

If the requested response is in the cache:

- The proxy sends the cached response to the client.

Proxy as a Client

If the response is not in the cache:

- The proxy sends a request to the target server.
- It receives the response.
- It then acts as a server and sends the response to the client.

Proxy Server Location

Proxy servers are normally located near the client.

A hierarchy may exist at different levels:

- 1 **Client Computer**

- Small local cache

- 2 **Company or Campus LAN**

- Reduces traffic entering and leaving the LAN

- 3 **ISP Network**

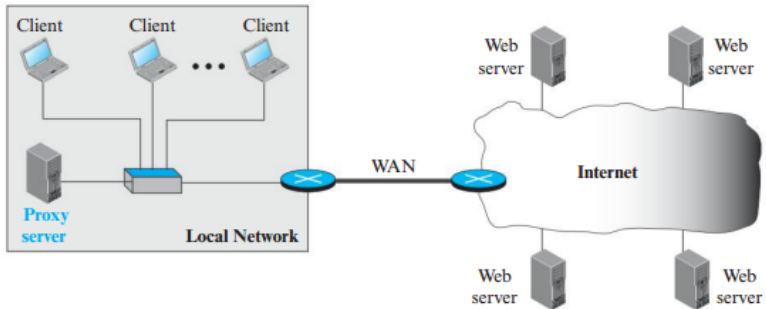
- Reduces traffic entering and leaving the ISP network

Example 2.10: Proxy Server in a Local Network

A proxy server is installed in a campus or company network.

- Client requests are first sent to the proxy.
- If the page is cached:
 - The proxy sends it directly to the client.
- If the page is not cached:
 - The proxy contacts the web server on the Internet.

Figure 2.16 *Example of a proxy server*



Example 2.10: Response Through the Proxy

When the requested page is not cached:

- 1 The proxy acts as a client.
- 2 It requests the page from the Internet web server.
- 3 The response returns to the proxy.
- 4 The proxy makes a copy.
- 5 The copy is stored in the cache.
- 6 The response is sent to the requesting client.

Future Benefit

Later clients can receive the cached copy without contacting the original server.

Important Question

How long should a response remain in the proxy cache?

Several strategies can be used:

- Consider how frequently a website changes.
- Example:
 - A news page may change every morning.
 - The proxy can keep the page until the next expected update.
- Use headers containing:
 - Last modification date
 - Last modification time

Keeping Cached Content Fresh

The proxy can use modification information to estimate validity.

- Recently modified pages may need frequent checking.
- Stable pages can remain cached longer.
- HTTP headers can help determine:
 - When the resource was last changed
 - Whether the cached copy is still valid

Goal

Balance fast access with delivery of up-to-date content.

Limitation

HTTP by itself does not provide security.

HTTP can be run over a security layer.

HTTP + Security Layer $\Rightarrow$ HTTPS

HTTPS provides:

- Confidentiality
- Client authentication
- Server authentication
- Data integrity

HTTP Topic Summary

- HTTP follows a request-response model.
- HTTP uses reliable, connection-oriented TCP.
- Connections may be:
 - Nonpersistent
 - Persistent
- HTTP defines request and response message formats.
- Conditional requests avoid unnecessary data transfer.
- Cookies add state to the originally stateless Web.
- Proxy servers provide web caching.
- HTTPS provides security for web communication.